Le Langage C

Dr. Mohamed SBAI

Force du langage C

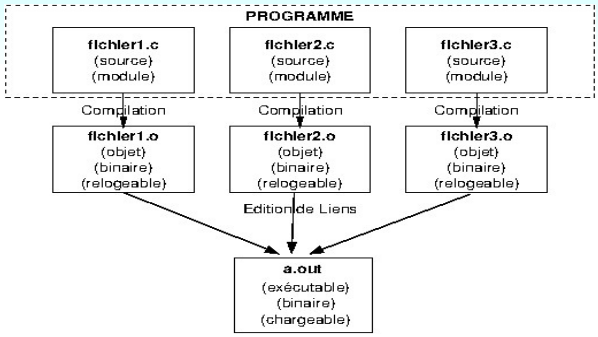
Le langage C est un langage de bas niveau dans le sens où il permet l'accès à des données que manipulent les ordinateurs (bits, octets, adresses) et qui ne sont pas souvent disponibles à partir de langages évolués tels que Fortran, Pascal ou ADA.

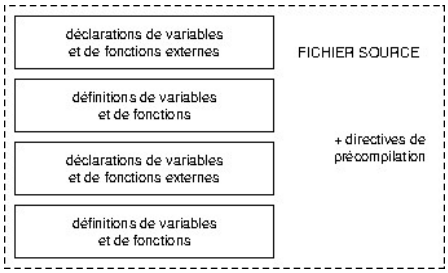
Le langage C a été conçu pour l'écriture de systèmes d'exploitation et du logiciel de base. Plus de 90% du noyau du système UNIX est écrit en langage C. Le compilateur C lui-même est écrit en grande partie en langage C ou à partir d'outils générant du langage C. Il en est de même pour les autres outils de la chaîne de compilation (assembleur, éditeur de liens, pré-processeur). De plus, tous les utilitaires du système sont écrits en C (shell, outils).

Il est cependant suffisamment général pour permettre de développer des applications variées de type scientifique ou encore pour l'accès aux bases de données. Par le biais des bases de données, il est utilisé dans les applications de gestion. De nombreux logiciels du domaine des ordinateurs personnels, tels que Microsoft Word ou Microsoft Excel, sont eux-aussi écrits à partir de langage C ou de son successeur orienté objet.

Le compilateur de langage C se limite aux fonctionnalités qui peuvent être traduites efficacement en instructions machine.

Compilation d'un programme





Fichier source

Le compilateur lit ce que génère le pré-processeur et crée les lignes d'assembleur correspondantes.

Le compilateur lit le texte source une seule fois du début du fichier à la fin. Cette lecture conditionne les contrôles qu'il peut faire, et explique pourquoi toute variable ou fonction doit être déclarée avant d'être utilisée.

STRUCTURE DE BASE

Délimiteurs de commentaires
- /* et */ pour des commentaires qui se prolongent sur plusieurs lignes
- // pour un commentaire en fin de ligne (unique)

Le point d'entrée de tout programme est repéré par la procédure **main()**

```
/* programme principal
-------------------------------------- */
void main ( )
{
   déclaration var 1 ; //pression
   ...
   déclaration var n ;

   instruction 1 ;
   ...
   instruction m ;
}
```

principal.cpp

Début du bloc

Le **corps** du programme *(ou d'un bloc d'instructions)* est toujours délimité par les accolades { et }

Fin du bloc

Un bloc d'instructions est divisé en 2 zones :
- zone de **déclaration,**
- **séquence d'instructions**,

Toutes les **déclarations** (*variable, type, sous-programme)* et toutes les instructions doivent se terminer par un **(;)**

LES IDENTIFICATEURS

Identificateur : Nom donné à une variable, un type, un sous-programme …

- **Doit commencer par une lettre ou un _**
- **Ne doit par être un mot clef du langage C (ces mots clés sont les suivants :**

type des données :
char const double float int long short signed unsigned void volatile

classes d'allocation : auto extern register static

Constructeurs : enum struct typedef union

instructions de boucle : do for while

Sélections : case default else if switch

ruptures de séquence : break continue goto return

Divers: asm entry fortran sizeof

- **Pas d'équivalence entre majuscule et minuscule**
  - **Mots clefs du C en minuscule**

DECLARATIONS ET MODULES

REGLES DE COMPILATION

- **Règle 1** : toute **variable** ou **constante** doit être **déclarée** avant de pouvoir être utilisée
- **Règle 2** : tout **type défini par l'utilisateur** doit être **déclaré** avant de pouvoir être utilisé.
- **Règle 3** : tout **sous-programme** *(procédure ou fonction)* doit être **déclaré** avant de pouvoir être appelé.

INTERET DE LA DECLARATION

- permet de vérifier **dès la compilation** que :
  - l'utilisation d'une variable ou d'un type est cohérent (affectation de valeurs compatibles avec le type ou la structure de la variable),
  - l'appel d'un sous-programme est cohérent avec la manière dont celui-ci est spécifié (notamment, bon nombre et bon type d'arguments)
- permet de faire de la compilation séparée ⇨ gain de temps
  - ⇨ décomposition d'un programme en un **fichier principal** (contenant `main()`) et de multiples **modules**

DECLARATIONS ET MODULES

MODULE (à voire plus loin)

- En C, un **module** est le groupement de deux fichiers :
  - un fichier avec l'extension **.h** *(« **h**eader file » ⇔ fichier entête)* qui contient la **déclaration** des sous-programmes et des types,
  - un fichier avec l'extension **.c** *(fichier source)* qui contient les instructions (ou **définition**) des sous-programmes.

- En général, le fichier « entête » et le fichier « source » porte le même nom ⇨ celui du module :

  Module *Interface* ⇔ fichiers *Interface.h* et *Interface.cpp*

- Pour utiliser un sous-programme ou un type situé dans le module *nom* depuis un programme situé dans un autre fichier, il faut satisfaire les règles 2 et 3. Pour cela, il suffit de placer au début de ce fichier la directive de compilation suivante :

```
#include "nom.h"
```


TYPES SIMPLES PREDEFINIS

Type	Algorithme	C
entier simple signé	ENTIER	**int**
entier long signé	Entier	**long**
réel simple précision	REEL	**float**
réel double précision	REEL	**double**
caractère	CHARACTERE	**char**
pointeur	POINTEUR	*****
non typé	-	**void**

Remarques :

- **Attribut `unsigned` pour les types à valeur entière *(char, int, long)***
- **Un entier égal à 0 est associé à la valeur booléenne *`false` (et ≠0, à la valeur `true`)***
- **L'instruction `typedef` permet de renommer un type :**

  `typedef` *ancien_nom nouveau_nom*;

DECLARATION D'UNE VARIABLE

id_type *id_var* = *valeur_initiale* ;

Obligatoire — *Facultatif*

- ***id_var*** : **nom (ou *identificateur*) de la variable,**
- ***id_type*** : **type de la variable ⇨ permet de lui allouer une place en mémoire :**
  - *réserve le nombre d'octet adéquat,*
  - *fixe sa position ⇔ son **adresse mémoire***
- ***valeur_initiale*** : **Permet éventuellement de donner une valeur initiale à la variable**
- **Exemple :**

```
int a = -12 ;
unsigned long Compteur = 0 ;
double Volume ;
char Lettre = 'a' ;
```

- **Correspond à la déclaration d'une variable dont :**
  - la **création** et la **destruction** sont gérées automatiquement par le compilateur,
  - la **durée de vie** s'étend de l'endroit où elle est déclarée jusqu'à la fin du bloc au sein duquel elle a été déclarée.
  - la **portée** (ou *visibilité*) est limitée à cette même zone.

LES OPERATEURS

TYPE	UNAIRES	BINAIRES	TERNAIRES
arithmétique	`-` : négation `++` : incrémentation de 1 `--` : décrémentation de 1	`+` : Addition `-` : Soustraction `*` : multiplication `/` : division `%` : modulo `+=, -=, *=, /=, %=` : version condensée	
logique	`!` : négation	`==` : égal `!=` : différent `> >=` : supérieur, supérieur ou égal `< <=` : inférieur, inférieur ou égal `&&` : et logique `||` : ou logique	`? :` : test conditionnel
bit à bit	`~` : négation binaire (complément à un)	`&` : et binaire `|` : ou binaire `^` : ou exclusif binaire `<<` : décalage à gauche `>>` : décalage à droite `&= |= ^= <<= >>=` : version condensée	
divers	`(type)` : conversion de type `sizeof(...)` : taille de l'argument en nombre d'octets	`=` : Affectation	

`i++` ⇔ `i=i+1`

`i--` ⇔ `i=i-1`

Utiliser pour les tests logiques

Utiliser pour affecter une valeur à une variable

DECLARATION D'UNE CONSTANTE

- **CONSTANTE : Entité dont la valeur ne peut pas être modifiée durant l'exécution du programme,**

Mot clef C → `const` *id_type* ***id_const = valeur*** ;
Obligatoire

- **Exemple :**

```
const int a = 12 ;
const PRECISION = 5.0E-6 ;
const double VOLUME_MAX = 20.0 ;
const char Lettre = 'a' ;
```

- **Lorsqu'un ensemble de constante ont une sémantique commune, il est préférable de les remplacer par un type énumération :**

```
const int NORD = 0 ;
const int SUD  = 1 ;
const int EST  = 2 ;
const int OUEST= 3 ;
```
⇔ `enum Orientation { NORD,SUD,EST,OUEST } ;`

LE TYPE « ENUMERATION »

- **Déclaration d'un type « énumération»**

enum *identificateur* { $label_1$, $label_2$, ... , $label_i$, ..., $label_n$} ;

Nom du type

éléments de la liste d'énumération
⇕
constantes entières
⇩
Par défaut, valeurs numérotées à partir de 0 avec un pas de progression de 1

➡ **Exemples :**

```
enum Logical {FALSE, TRUE} ;

enum Feux_Tricolore {ROUGE, ORANGE, VERT} ;

enum Les_Mois { JANVIER=1, FEVRIER, MARS, AVRIL, MAI, JUIN, JUILLET, AOUT,
                SEPTEMBRE, OCTOBRE, NOVEMBRE, DECEMBRE} ;

enum Orientation {NORD, SUD, EST, OUEST} ;

enum Bouton { PLUS=100, MOINS, FICHIER, SIMULER } ;
```

➡ **Utilisation d'un type « énumération»**

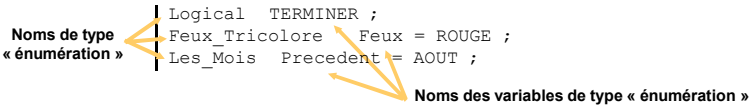
- **permet de définir des listes de constantes,**
- **souvent utilisées en association avec la structure d'aiguillage `switch`() pour libeller chaque `case`.**

LE TYPE « ENUMERATION »

Déclaration d'une variable de type « énumération»

- S'effectue d'une manière totalement identique à celle d'une variable d'un type prédéfini :

Exemples :



- Seuls les opérateurs de comparaisons (>,<, ==, !=,<=,>=) sont directement utilisables,

```
Les_Mois  Courant;
Les_Mois  Suivant = OCTOBRE ;
        ↳  if  (Courant < Suivant) ...
```

- Les opérateurs arithmétiques (+,-, *,/,%,++,--.) sont utilisables seulement après conversion vers un type entier (`char, int, long`)

```
Les_Mois  Courant = SEPTEMBRE ;
Les_Mois Suivant ;

  ↳  Suivant = Courant + 1 ;                    incorrect
  ↳  Suivant = (Les_Mois)((int)Courant + 1) ;   correct
```


Les commentaires

Comme tout langage évolué, le langage C autorise la présence de commentaires dans vos programmes source. Il s'agit de textes explicatifs destinés aux lecteurs du programme et qui n'ont aucune incidence sur sa compilation.

Ils sont formés de caractères quelconques placés entre les symboles **/*** et ***/** ou **//**. Ils peuvent apparaître à tout endroit du programme où un espace est autorisé. En général, cependant, on se limitera à des emplacements propices à une bonne lisibilité du programme.

Voici quelques exemples de commentaires :

sur une lignes

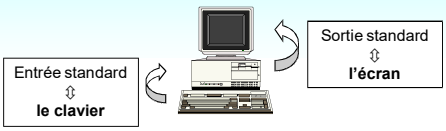
// programme de calcul de racines carrées

sur plusieurs lignes

```
/* de programme source
========================================
commentaire quelque peu esthétique *
et encadré, pouvant servir, *
par exemple, d'en-tête de programme *
======================================== */
```

LES INSTRUCTIONS

INSTRUCTIONS D'ENTREE/SORTIE



- Nécessite l'inclusion d'une librairie en début de programme,
  `#include <stdio.h>`
- Instruction d'affichage à l'écran `printf ( "format", var1, var2,…, varn);`
- Instruction de saisie au clavier, `scanf ( "format", &var1) ;`

spécificateurs de format

%d : entier en notation décimale
%o : entier en notation octale
%x : entier en notation hexadécimale
%u : entier en notation décimal non signée
%p : pointeur
%f : réel en représentation flottante
%e : réel en représentation exposant
%c : caractère
%s : chaîne de caractères
%l : élément long (entier ou réel)

Plus sur printf() et scanf()

Les fonctions **printf()** et **scanf()** transforment des objets d'une représentation à partir d'une chaîne de caractères (vision humaine) en une représentation manipulable par la machine (vision machine), et vice et versa.

Pour réaliser ces transformations ces fonctions sont guidées par des formats qui décrivent le type des objets manipulés (vision interne) et la représentation en chaîne de caractères cible (vision externe).

Par exemple, un format du type **%x** signifie d'une part que la variable est du type entier et d'autre part que la chaîne de caractères qui la représente est exprimée en base 16 (hexadécimal).

Pour printf(), un format est une chaîne de caractères dans laquelle sont insérés les caractères représentant la ou les variables à écrire.

Pour scanf(), un format est une chaîne de caractères qui décrit la ou les variables à lire.
Pour chaque variable, un type de conversion est spécifié. Ce type de conversion est décrit par les caractères qui suivent le caractère "%".

résumé des déclarations de variables et des formats

23

déclaration	lecture	écriture	format externe
int i ;	scanf ("%d",&i);	printf ("%d",i);	décimal
int i ;	scanf ("%o",&i);	printf ("%o",i);	octal
int i ;	scanf ("%x",&i);	printf ("%x",i);	hexadécimal
unsigned int i ;	scanf ("%u",&i);	printf ("%u",i);	décimal
short j ;	scanf ("%hd",&j);	printf ("%d",j);	décimal
short j ;	scanf ("%ho",&j);	printf ("%o",j);	octal
short j ;	scanf ("%hx",&j);	printf ("%x",j);	hexadécimal
unsigned short j ;	scanf ("%hu",&j);	printf ("%u",j);	décimal
long k;	scanf ("%ld",&k);	printf ("%ld",k);	décimal
long k;	scanf ("%lo",&k);	printf ("%lo",k);	octal
long k;	scanf ("%lx",&k);	printf ("%lx",k);	hexadécimal
unsigned long k;	scanf ("%lu",&k);	printf ("%lu",k);	décimal
float l ;	scanf ("%f",&l);	printf ("%f",l);	point décimal
float l ;	scanf ("%e",&l);	printf ("%e",l);	exponentielle
float l ;		printf ("%g",l);	la plus courte des deux
double m;	scanf ("%lf",&m);	printf ("%f",m);	point décimal
double m;	scanf ("%le"&m);	printf ("%e",m);	exponentielle
double m;		printf ("%g",m);	la plus courte
long double n;	scanf ("%Lf"&n);	printf ("%Lf",n);	point décimal
long double n;	scanf ("%Le"&n);	printf ("%Le",n);	exponentielle
long double n;		printf ("%Lg",n);	la plus courte
char o;	scanf ("%c",&o);	printf ("%c",o);	caractère
char p [10];	scanf ("%9s",p); scanf ("%9s",&p[0]);	printf ("%s",p);	chaîne de caractères

INSTRUCTIONS D'ENTREE/SORTIE

- **Exemple**

```
#include  <stdio.h>

void main ()
{
   int a = 10 ;
   int b ;

   printf ("Donnez b  : " ) ;
   scanf ("%d", &b )  ;
   printf ("Resultat  : %d*%d=%d \n",a,b,
            a*b )  ;
   printf ("Fin")  ;
}
```

```
Donnez  b : 5
Resultat : 10*5=50
Fin
```

un certain nombre d'abréviations est aussi disponible :

'\a' : alert (sonnerie, BEL) ;
'\b' : backspace (BS) ;
'\f' : saut de page (FF) ;
'\n' : saut de ligne (LF) ;
'\r' : retour chariot (CR) ;
'\t' : tabulation horizontale (HT) ;
'\v' : tabulation verticale (VT) ;

Lectures multiples avec scanf()

```
# inc lude < s t d i o . h>
i n t main ( i n t a rgc , char  a rgv [ ] )
 {
   i n t i = 1 0 ;
   f l o a t l = 3 . 1 4 1 5 9 ;
   char p [ 5 0 ] = " Bonjour " ;
   p r i n t f ( "%d b o n j o u r %f %s \ n " , i , l , p ) ;
   s c a n f ( "%d%f%49s " , &i , &l , p ) ;
   p r i n t f ( " Apres l e c t u r e au c l a v i e r : %d %f %s \ n " , i , l , p ) ;
   r e turn 0 ;
 }
```

DONNÉES EN ENTRÉE

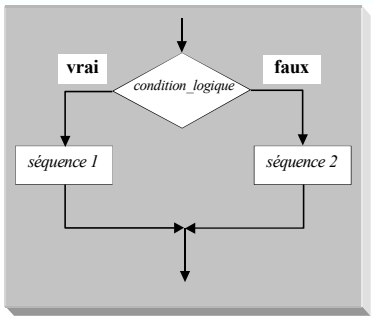
```
        23      6.55957                 salut
```

DONNÉES ÉCRITES SUR LE FICHIER STANDARD DE SORTIE

```
        10      3.141590        Bonjour
Apres lecture au clavier : 23      6.559570        salut
```

INSTRUCTIONS CONDITIONNELLES

Structure conditionnelle avec alternative : if



C/C++

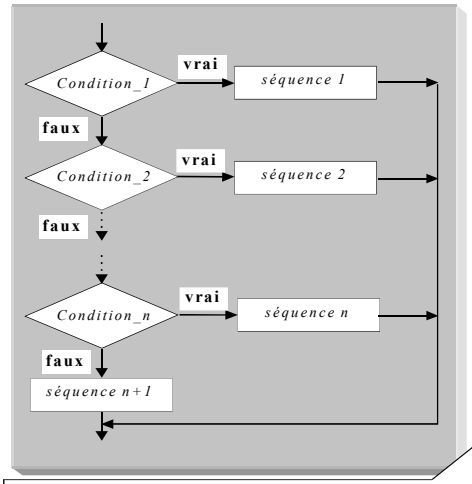
```
if (condition_logique)
{
    séquence 1 ;
}
else
{
    séquence 2 ;
}
```

- la condition logique doit toujours être entre parenthèses (…),
- peuvent être imbriquées,
- si la séquence d'instruction ne comporte qu'une seule instruction, les accolades { et } peuvent éventuellement être omises,
- si elle n'est pas nécessaire, l'alternative `else` peut être omise.

Un else se rapporte toujours au dernier if rencontré auquel un else n'a pas encore été attribué.

INSTRUCTIONS CONDITIONNELLES

Structure de sélection avec alternative multiple sur conditions



C/C++

```
if (condition_1)
{
    séquence 1 ;
}
else if (condition_2)
{
    séquence 2 ;
}
    ⋮
    ⋮
else if (condition_n )
{
    séquence n ;
}
else
{
    séquence n+1 ;
}
```

- permet « d'aiguiller » l'exécution du programme vers le bloc d'instructions dont la *condition logique* d'entrée est vérifiée,
- le dernier **else** n'est pas obligatoire mais recommandé.

Exemple de if

Voici un exemple d'utilisation de if imbriqués. Il s'agit d'un programme de facturation avec remise. Il lit en donnée un simple prix hors taxes et calcule le prix TTC correspondant (avec un taux de TVA constant de 18,6 %). Il établit ensuite une remise dont le taux dépend de la valeur

ainsi obtenue, à savoir :

- 0 % pour un montant inférieur à 1 000 F
- 1 % pour un montant supérieur ou égal à 1 000 F et inférieur à 2 000 F
- 3 % pour un montant supérieur ou égal à 2 000 F et inférieur à 5 000 F
- 5 % pour un montant supérieur ou égal à 5 000 F

Solution de l'exemple de if

```
#include <stdio.h>
#define TAUX_TVA 18.6
main()
{
    double ht, ttc, net, tauxr, remise ;
    printf("donnez le prix hors taxes : ") ;
    scanf ("%lf", &ht) ;
    ttc = ht * ( 1. + TAUX_TVA/100.) ;
    if ( ttc < 1000.)
            tauxr = 0 ;
    else if ( ttc < 2000 )
            tauxr = 1. ;
    else if ( ttc < 5000 )
            tauxr = 3. ;
    else
            tauxr = 5. ;
    remise = ttc * tauxr / 100. ;
    net = ttc - remise ;
    printf ("prix ttc %10.2lf\n", ttc) ;
    printf ("remise %10.2lf\n", remise) ;
    printf ("net à payer %10.2lf\n", net) ;
}
```

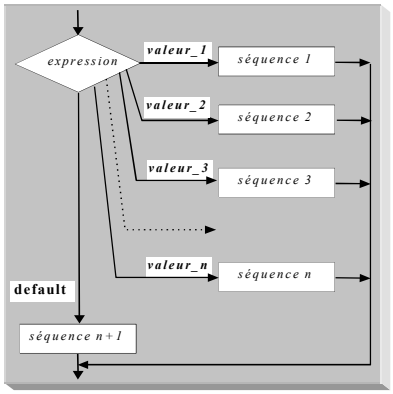
EXECUTION

```
donnez le prix hors taxes : 500
prix ttc 593.00
remise 0.00
net à payer 593.00

donnez le prix hors taxes : 4000
prix ttc 4744.00
remise 142.32
net à payer 4601.68
```

INSTRUCTIONS CONDITIONNELLES

Structure de sélection avec alternative multiple sur valeurs



C/C++

```
switch ( expression )
{
   case valeur_1 : séquence 1 ;
                   break ;
   case valeur_2 : séquence 2 ;
                   break ;
   case valeur_3 : séquence 3 ;
                   break ;
            ⋮
   case valeur_n : séquence n ;
                   break ;
   default       : séquence n+1 ;
}
```

- permet « d'aiguiller » l'exécution du programme vers le bloc d'instructions dont la *condition* d'entrée (valeur associée au `case`) correspond au résultat de l'*expression* (située dans le `switch`),
- la valeur de *expression* doit être un type discret (`char`, `int`, `long`) ou un type énumération,
- l'étiquette `default` n'est pas obligatoire mais recommandée,
- l'instruction `break` permet de sortir de la structure `switch` après l'exécution des instructions associées à un `case` donné,
- Pour exécuter la même séquence d'instructions pour des `case` différents *(conjonction de valeurs)*, il suffit d'écrire ces `case` consécutivement et de n'associer le `break` qu'au dernier `case` de la liste.

Exemple de switch

```
main()
{
        int n ;

        printf ("donnez un entier : ") ;
        scanf ("%d", &n) ;
        switch (n)
        {
            case 0 : printf ("nul\n") ;
                     break ;
            case 1 : printf ("un\n") ;
                     break ;
            case 2 : printf ("deux\n") ;
                     break ;
            default : printf ("grand\n") ;
        }
        printf ("au revoir\n") ;
}
```

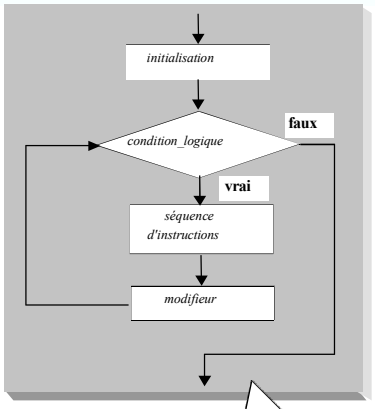
Exécution

```
donnez un entier : 2
deux
au revoir

donnez un entier : 25
grand
au revoir
```

INSTRUCTIONS DE REPETITION

Structure de répétition sur compteur



Pas > 0

```
for ( var=début ; var<=fin ; var=var+pas )
{
        séquence d'instructions ;
}
```

Pas < 0

C/C++

```
for ( var=début ; var>=fin ; var=var+pas )
{
        séquence d'instructions ;
}
```

- à utiliser lorsque le nombre de répétition est connu d'avance (1 à *n* fois),
- doit être interprétée comme une structure `while`

Exemple de for

```
main()
{
        int i ;

        for ( i=1 ; i<=5 ; i++ )
        {
                printf ("bonjour ") ;
                printf ("%d fois\n", i) ;
        }
}
```

Exécution

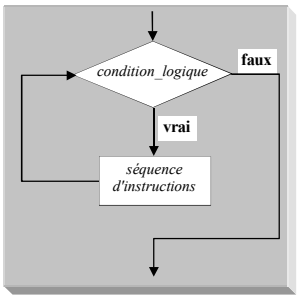
bonjour 1 fois
bonjour 2 fois
bonjour 3 fois
bonjour 4 fois
bonjour 5 fois

La ligne : for (i=1 ; i<=5 ; i++)

comporte en fait trois expressions. La première est évaluée (une seule fois) avant d'entrer dans la boucle. La deuxième conditionne la poursuite de la boucle. Elle est évaluée **avant chaque** parcours. La troisième, enfin, est évaluée à la fin de chaque parcours.

INSTRUCTIONS DE REPETITION

- **Structure de répétition sur condition :** ***tant que (...) faire {...}***



C/C++

```
...
while( condition_logique )
{
    séquence d'instructions ;
}
...
```

- utilisée lorsque le nombre de répétition est a priori inconnu (0 à ? fois),
- toujours initialiser les variables intervenant dans la condition logique,
- la séquence d'instructions répétée doit obligatoirement contenir une action qui modifie la valeur de la condition logique,

TD 1 : Exercice 4.4

Exemple de while()

```
main()
{
        int n, som ;

        som = 0 ;
        while (som<100)
        {
                printf ("donnez un nombre : ") ;
                scanf ("%d", &n) ;
                som += n ;
        }
        printf ("somme obtenue : %d", som) ;
}
```

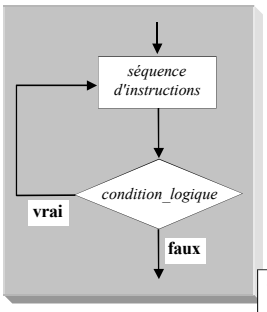
Exécution

donnez un nombre : 15
donnez un nombre : 25
donnez un nombre : 12
donnez un nombre : 60
somme obtenue : 112

répète l'instruction qui suit (ici un bloc) tant que la condition mentionnée est vraie, comme le ferait do... while. En revanche, cette fois, la condition de poursuite est examinée **avant chaque parcours de la boucle et non après. Ainsi, contrairement à ce qui se** passait avec do... while, une telle boucle peut très bien n'être parcourue aucune fois si la condition est fausse dès qu'on l'aborde (ce qui n'est pas le cas ici).

INSTRUCTIONS DE REPETITION

- **Structure de répétition sur condition : *faire {...} tant que (...)***



C/C++

```
do
{
    séquence d'instructions ;
}
while ( condition_logique ) ;
```

- utilisée lorsque le nombre de répétition est a priori inconnu (1 à ? fois),
- l'initialisation des variables intervenant dans la condition logique n'est pas indispensable,
- la séquence d'instructions répétée doit obligatoirement contenir une action qui modifie la valeur de la condition logique,

TD 1 : Exercice 4.6

Exemple de do ... while()

```
main()
{
        int n ;
        do
        {
                printf ("donnez un nb >0 : ") ;
                scanf ("%d", &n) ;
                printf ("vous avez fourni %d\n", n) ;
        }
        while (n<=0) ;
printf ("réponse correcte") ;
}
```

Exécution

```
donnez un nb >0 : -3
vous avez fourni -3
donnez un nb >0 : -9
vous avez fourni –9
donnez un nb >0 : 12
vous avez fourni 12
réponse correcte
```

répète l'instruction qu'elle contient (ici un bloc) tant que la condition mentionnée (n<=0) est vraie (c'est-à-dire, en C, non nulle). Autrement dit, ici, elle demande un nombre à l'utilisateur (en affichant la valeur lue) tant qu'il ne fournit pas une valeur positive.

LES TABLEAUX

TABLEAUX A UNE DIMENSION

TABLEAU : **Structure de données de dimension finie dont les éléments sont tous de même type**

➡ **Déclaration d'un tableau :**

type nom **[** *n* **]** = { val_1, val_2, ..., val_n } ;

- *type* : Type des éléments
- *nom* : Nom du tableau
- *n* : Nombre d'éléments
- val_1, val_2, ..., val_n : Valeurs initiales *(facultatif)*

➡ **Exemple :**

```
int tab[5] ;                    (tableau de 5 entiers)
float Temperature[3] = {23.0, 10.4, 34.1 };   (tableau de 3 réels initialisés)
unsigned char PDU[] = { 0x1C, 0xFE, 0x2A, 0x10 } ;   (dimension ⇨ 4)
double  Cond_Init[6] = { 5.0E-3, 1.0e-8 } ;   (⇨ 4 derniers éléments initialisés à 0)
```

➡ **Remarques :**

- en C, les **tableaux commencent à l'indice 0** (⇨ *indice du 1er élément : 0*),
- pas de vérification de débordement,
  Exemple pour remplir le tableau de 10 entier:

```
For(i=0;i<10;i++)
  scanf("%d",&tab[ i ]);
```

CHAINE DE CARACTERES

CHAINE DE CARACTERE :

Tableau de caractères où le dernier caractère est le caractère \0 *(délimiteur de chaîne).*

"Chaine" ⇨ | C | h | a | i | n | e | \0 | (*Attention* : 'a' ≠ "a")

➡ **Déclaration d'une chaîne de caractère :**

char *nom* **[** *n* **]** = "*texte*" ;

- *nom* : Nom de la chaîne
- *n* : Dimension = nombre de caractères effectifs + 1 *(pour le \0)*
- *texte* : texte initial *(facultatif)*

➡ **La manipulation des chaînes de caractères au moyen de fonctions fournies dans la librairie <string.h>**

Généralités sur les fonctions portant sur des chaînes

41

La fonction strlen: La fonction strlen fournit en résultat la longueur d'une chaîne dont on lui a transmis l'adresse en argument.
strlen ("bonjour") vaudra 7 ; de même, avec : char * adr = "salut" ; strlen (adr) vaudra 5.

La fonction strcat: strcat (but, source); Cette fonction recopie la seconde chaîne (source) à la suite de la première (but), après en avoir effacé le caractère de fin.

La fonction strncat: strncat (but, source, lgmax); travaille de façon semblable à strcat en offrant en outre un contrôle sur le nombre de caractères qui seront concaténés à la chaîne d'arrivée (but).

La fonction: strcmp (chaîne1, chaîne2); compare deux chaînes dont on lui fournit l'adresse et elle fournit une valeur entière définie comme étant :
- positive si chaîne1 > chaîne2 (c'est-à-dire si chaîne1 arrive après chaîne2, au sens de l'ordre défini par le code des caractères) ;
- nulle si chaîne1 = chaîne2 (c'est-à-dire si ces deux chaînes contiennent exactement la même suite de caractères) ;
- négative si chaîne1 < chaîne2.

Par exemple (quelle que soit l'implémentation) : strcmp ("bonjour", "monsieur") est négatif et : strcmp ("paris2", "paris10") est positif.

Généralités sur les fonctions portant sur des chaînes

La fonction : strncmp (chaîne1, chaîne2, lgmax); travaille comme strcmp mais elle limite la comparaison au nombre maximal de caractères indiqués par l'entier lgmax.
Par exemple : strncmp ("bonjour", "bon", 4) est positif tandis que : strncmp ("bonjour", "bon", 2) vaut zéro.

La fonction : strcpy (but, source); recopie la chaîne située à l'adresse source dans l'emplacement d'adresse destin. Là encore, il est nécessaire que la taille du second emplacement soit suffisante pour accueillir la chaîne à recopier, sous peine d'écrasement intempestif. Cette fonction fournit comme résultat l'adresse de la chaîne but.

La fonction : strncpy (but, source, lgmax); procède de manière analogue à strcpy, en limitant la recopie au nombre de caractères précisés par l'expression entière lgmax.

La fonction : strchr (chaîne, caractère); recherche, dans chaîne, la première position où apparaît le caractère mentionné.

La fonction : strrchr (chaîne, caractère); réalise le même traitement que strchr, mais en explorant la chaîne concernée à partir de la fin. Elle fournit donc la dernière occurrence du caractère mentionné.

La fonction : strstr (chaîne, sous-chaîne); recherche, dans chaîne, la première occurrence complète de la sous-chaîne mentionnée.

TABLEAUX MULTIDIMENSIONNELS

TABLEAU MULTIDIMENSIONNEL :

création de tableaux à *p* dimensions d_1, d_2, d_3,…,d_p en juxtaposant *p* opérateurs d'indexation [ ].

➡ **Déclaration d'un tableau à *p* dimensions :**

$$type \;\; nom \;\; [n_1][n_2]\ldots[n_{p-1}][n_p] = \{ val_1, val_2, \ldots, val_k \} ;$$

- Type des éléments
- Nom du tableau
- Nombre d'éléments sur la $j^{ème}$ dimension
- Valeurs initiales *(facultatif)* $k = n_1 \times n_2 \times \ldots \times n_{p-1} \times n_p$

➡ **Remarque** : l'initialisation d'un tableau multidimensionnel est réalisé **ligne par ligne**

Exemple pour remplir le tableau de 10 lignes et 6 colonnes on a besoin de **deux indices i et j qui sont des variables de type entier** :

```
For(i=0;i<10;i++)
      For(j=0;j<10;j++)
            scanf("%d",&tab[ i ] [ j ]);
```

Les enregistrements : le type structure

Nous avons déjà vu comment le tableau permettait de désigner sous un seul nom un ensemble de valeurs de même type, chacune d'entre elles étant repérée par un indice.

La structure, quant à elle, va nous permettre de désigner sous un seul nom un ensemble de valeurs pouvant être de types différents. L'accès à chaque élément de la structure (nommé *champ) se fera, cette fois, non plus par une indication de position, mais par son nom au sein de* la structure.

Une structure est donc une variable composée de plusieurs champs qui sert à représenter un objet réel ou un concept.
Par exemple une voiture peut être représentée par les renseignements suivants : la marque, la couleur, l'année, . . .
Le langage C permet de définir des modèles de structures comme les autres langages évolués.

Déclaration d'une structure

Voyez tout d'abord cette déclaration :

```
struct enreg
{
        int   numero ;
        int   qte ;
        float prix ;
} ;
```

Celle-ci définit un **modèle de structure mais ne réserve pas de variables correspondant à cette** structure. Ce modèle s'appelle ici **enreg** et il précise le nom et le type de chacun des **champs** constituant la structure (numero, qte et prix). Une fois un tel modèle défini, nous pouvons déclarer des variables du type correspondant (souvent, nous parlerons de structure pour désigner une variable dont le type est un modèle de structure).

Par exemple : struct enreg art1 ;
réserve un emplacement nommé art1 « de type enreg » destiné à contenir deux entiers et un flottant.

De manière semblable : struct enreg art1, art2 ;
réserve deux emplacements art1 et art2 du type enreg.

Utilisation d'une structure

En C, on peut utiliser une structure de deux manières :
- en travaillant individuellement sur chacun de ses champs ;
- en travaillant de manière globale sur l'ensemble de la structure.

Utilisation des champs d'une structure

Chaque champ d'une structure peut être manipulé comme n'importe quelle variable du type correspondant. La désignation d'un champ se note en faisant suivre le nom de la variable structure de l'opérateur « point » (.) suivi du nom de champ tel qu'il a été défini dans le modèle (le nom de modèle lui-même n'intervenant d'ailleurs pas). Voici quelques exemples utilisant le modèle enreg et les variables art1 et art2 déclarées de ce type.

art1.numero = 15 ; affecte la valeur 15 au champ numero de la structure art1.
printf ("%e", art1.prix) ; affiche, suivant le code format %e, la valeur du champ prix de la structure art1.
scanf ("%e", &art2.prix) ; lit, suivant le code format %e, une valeur qui sera affectée au champ prix de la structure art2. Notez bien la présence de l'opérateur &.
art1.numero++ incrémente de 1 la valeur du champ numero de la structure art1.

Remarque : La priorité de l'opérateur « . » est très élevée, de sorte qu'aucune des expressions ci-dessus ne nécessite de parenthèses (voyez le tableau du chapitre 3, « Les opérateurs et les expressions en langage C »).

Utilisation d'une structure

Utilisation globale d'une structure

Il est possible d'affecter à une structure le contenu d'une structure définie à partir du **même modèle.**

Par exemple, si les structures art1 et art2 ont été déclarées suivant le modèle enreg défini précédemment, nous pourrons écrire :
art1 = art2 ; Une telle affectation globale remplace avantageusement :

```
art1.numero = art2.numero ;
art1.qte = art2.qte ;
art1.prix = art2.prix ;
```

Notez bien qu'une affectation globale n'est possible que si les structures ont été **définies avec le même nom de modèle ; en particulier, elle sera impossible avec des variables ayant une** structure analogue mais définies sous deux noms différents. L'opérateur d'affectation et, comme nous le verrons un peu plus loin, l'opérateur d'adresse & sont les seuls opérateurs s'appliquant à une structure (de manière globale).

Remarque : L'affectation globale n'est pas possible entre tableaux. Elle l'est, par contre, entre structures. Aussi est-il possible, en créant artificiellement une structure contenant un seul champ qui est un tableau, de réaliser une affectation globale entre tableaux.

Utilisation d'une structure

Initialisations de structures

On retrouve pour les structures les règles d'initialisation qui sont en vigueur pour tous les types de variables, à savoir :

● En l'absence d'initialisation explicite, les structures de classe statique sont, par défaut, initialisées à zéro ; celles possédant la classe automatique ne sont pas initialisées par défaut (elles contiendront donc des valeurs aléatoires).

● Il est possible d'initialiser explicitement une structure lors de sa déclaration. On ne peut toutefois employer que des constantes ou des expressions constantes et cela aussi bien pour les structures statiques que pour les structures automatiques,alors que, pour les variables scalaires automatiques, il était possible d'employer une expression quelconque (on retrouve là les mêmes restrictions que pour les tableaux).

Voici un exemple d'initialisation de notre structure art1, au moment de sa déclaration :
struct enreg art1 = { 100, 285, 2000 } ;

Vous voyez que la description des différents champs se présente sous la forme d'une liste de valeurs séparées par des virgules, chaque valeur étant une constante ayant le type du champ correspondant. Là encore, il est possible d'omettre certaines valeurs.

Utilisation d'une structure : typedef

Définir des synonymes avec typedef

La déclaration typedef permet de définir ce que l'on nomme en langage C des *types synonymes.*
A priori, elle s'applique à tous les types et pas seulement aux structures. C'est pourquoi nous commencerons par l'introduire sur quelques exemples avant de montrer l'usage que l'on peut en faire avec les structures. Exemples d'utilisation de **typedef**

La déclaration : **typedef int entier ;**
signifie que entier est synonyme de int, de sorte que les déclarations suivantes sont équivalentes : int n, p ; entier n, p ;

De même : **typedef int * ptr ;** signifie que ptr est synonyme de int *. Les déclarations suivantes sont équivalentes : int * p1, * p2 ; ptr p1, p2 ;

Notez bien que cette déclaration est plus puissante qu'une substitution telle qu'elle pourrait être réalisée par la directive #define. Nous n'en ferons pas ici de description exhaustive, et cela d'autant plus que son usage tend à disparaître. À titre indicatif, sachez, par exemple, qu'avec la déclaration : **typedef int vecteur [3] ;**
les déclarations suivantes sont équivalentes : int v[3], w[3] ; vecteur v, w ;

Typedef : Application aux structures

En faisant usage de typedef, les déclarations des structures art1 et art2 du précédemment peuvent être réalisées comme suit :

```
struct enreg
{
        int numero ;
        int qte ;
        float prix ;
} ;
typedef struct enreg s_enreg ;
s_enreg art1, art2 ;
```

ou encore, plus simplement :

```
typedef struct
{
        int numero ;
        int qte ;
        float prix ;
} s_enreg ;
s_enreg art1, art2 ;
```

Par la suite, nous ne ferons appel qu'occasionnellement à typedef, afin de ne pas vous enfermer dans un style de notations que vous ne retrouverez pas nécessairement dans les programmes que vous serez amené à utiliser.

Tableaux de structures

Voyez ces déclarations :

```
struct point
{
        char nom ;
        int x ;
        int y ;
} ;
struct point courbe [50] ;
```

La structure point pourrait, par exemple, servir à représenter un point d'un plan, point qui serait défini par son nom (caractère) et ses deux coordonnées. Notez bien que point est un nom de modèle de structure, tandis que courbe représente effectivement un tableau de 50 éléments du type point. Si i est un entier, la notation : **courbe[i].nom** représente le nom du point de rang i du tableau courbe. Il s'agit donc d'une valeur de type char. Notez bien que la notation : courbe.nom[i] n'aurait pas de sens.

De même, la notation : **courbe[i].x** désigne la valeur du champ x de l'élément de rang i du tableau courbe. Par ailleurs : **courbe[4]** représente la structure de type point correspondant au cinquième élément du tableau courbe.

Enfin **courbe est un identificateur de tableau, et, comme tel, désigne son adresse de début.** Là encore, voici, à titre indicatif, un exemple d'initialisation (partielle) de notre variable courbe, lors de sa déclaration :

```
struct point courbe[50]= { {'A', 10, 25}, {'M', 12, 28},, {'P', 18,2} };
```

Structures comportant d'autres structures

Supposez que, à l'intérieur de nos structures employe et courant définies, nous ayons besoin d'introduire deux dates : la date d'embauche et la date d'entrée dans le dernier poste occupé. Si ces dates sont elles-mêmes des structures comportant trois champs correspondant au jour, au mois et à l'année, nous pouvons alors procéder aux déclarations suivantes :

```
struct date
{
        int jour, mois, annee ;
} ;
                struct personne
                {
                        char nom[30] , prenom[20] ;
                        float heures [31] ;
                        struct date date_embauche ;
                        struct date date_poste ;
                } employe, courant ;
```

Vous voyez que la seconde déclaration fait intervenir un modèle de structure (date) précédemment défini.
La notation : **employe.date_embauche.annee** représente l'année d'embauche correspondant à la structure employe. Il s'agit d'une valeur de type int.

courant.date_embauche représente la date d'embauche correspondant à la structure courant. Il s'agit cette fois d'une structure de type date. Elle pourra éventuellement faire l'objet d'affectations globales comme dans :
courant.date_embauche = employe.date_poste ;